

CS 473 - MDP

Mobile Device Programming

© 2021 Maharishi International University

All course materials are copyright protected by international copyright laws and remain the property of the Maharishi International University. The materials are accessible only for the personal use of students enrolled in this course and only for the duration of the course. Any copying and distributing are not allowed and subject to legal action.



Maharishi International
University

CS 473 - MDP

Mobile Device Programming

MS.CS Program
Department of Computer Science
Renuka Mohanraj, Ph.D.



Lesson - 5

App Architecture and Dependency Injection

WHOLENESS OF THE LESSON

This lesson integrates app architecture, ViewModel state management, unit testing, and dependency injection with Hilt into one cohesive framework that strengthens clarity, reliability, and usability. *Science and Technology of Consciousness*: Just as knowledge is structured in consciousness, where the whole supports and gives meaning to each part, the structure of an app integrates architecture, testing, and UI so every layer contributes to the wholeness of the system.

Agenda

App Architecture

Unit Testing for ViewModel

Dependency injection with Hilt

More UIs

- RadioButton
- Checkbox

Mobile App Architecture

Learn about app architecture and how to use ViewModels, UI State, and StateFlow to build more complex apps.

App Architecture

Separation of concerns

- The separation of concerns design principle states that the app is divided into classes of functions, each with separate responsibilities.

Drive UI from a model

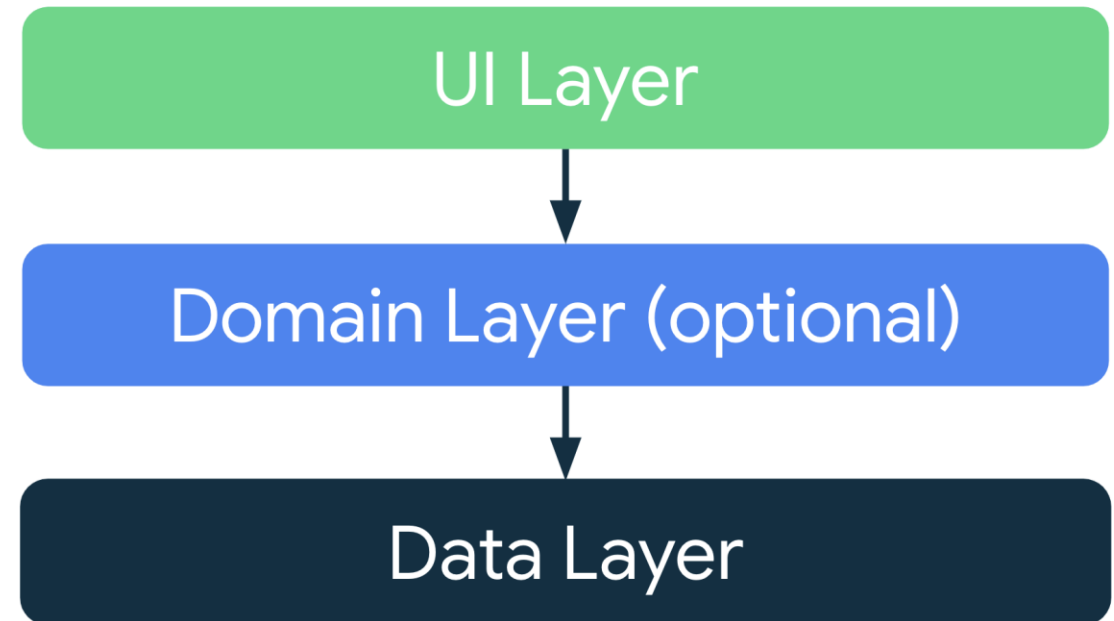
- The drive UI from a model principle states that you should drive your UI from a model, preferably a persistent model.
- Models are components responsible for handling the data for an app.
- Models are independent from the UI elements and app components in your app, so they're unaffected by the app's lifecycle and associated concerns.

Recommended app architecture

App should have at least two layers:

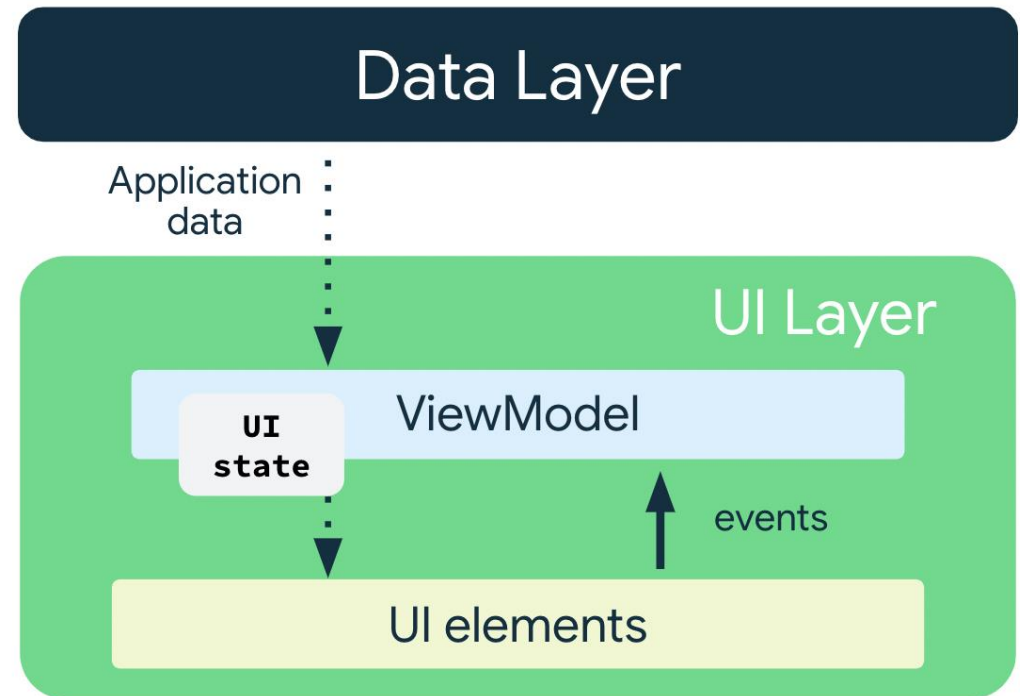
- UI layer
 - a layer that displays the app data on the screen but is independent of the data.
- Data layer
 - a layer that stores, retrieves, and exposes the app data.

You can add an additional layer called the domain layer to simplify and reuse the interactions between the UI and data layers.

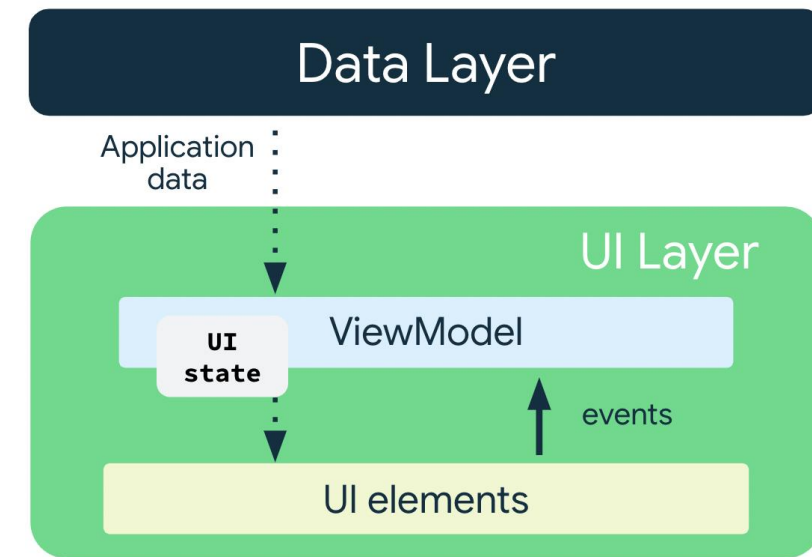


UI Layer

- The role of the UI layer, or presentation layer, is to display the application data on the screen.
- Whenever the data changes due to a user interaction, such as pressing a button, the UI should update to reflect the changes.



UI Layer

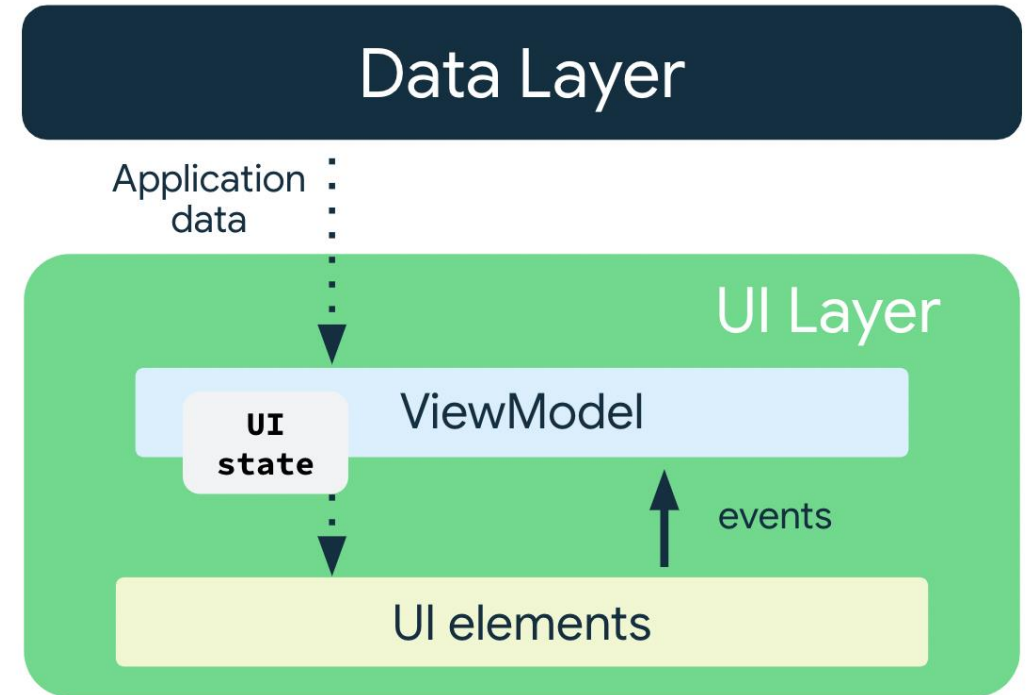


The UI layer is made up of the following components:

- UI elements
 - components that render the data on the screen. You build these elements using Jetpack Compose.
- State holders
 - components that hold the data, expose it to the UI, and handle the app logic. An example state holder is ViewModel.

ViewModel

- The ViewModel component holds and exposes the state the UI consumes.
- The UI state is application data transformed by ViewModel.
- ViewModel lets the app follow the architecture principle of driving the UI from the model.



Do not mix Logic with View

Separation of Concern

@Composable

```
fun MainScreen(modifier: Modifier = Modifier) {
```

```
    var counter = remember {  
        mutableIntStateOf(0)  
    }  
    var increment = { counter.value++ }  
    var decrement = { counter.value-- }
```

```
    Column(  
        modifier = modifier.fillMaxSize(),  
        //..  
    ) {  
        Button(onClick = { increment() }) {  
            Text(text = "Increment")  
        }  
        Text(text = "${counter.value}")  
        Button(onClick = { decrement() }) {  
            Text(text = "Decrement")  
        }  
    }  
}
```



`rememberSaveable{ }`

While **remember{ }** helps you retain state across recompositions, the state is not retained across configuration changes.

For this, you must use **rememberSaveable**.

rememberSaveable automatically saves any value that can be saved in a Bundle.

@Composable

```
fun MainScreen(modifier: Modifier = Modifier) {
```

```
    var counter = rememberSaveable {  
        mutableIntStateOf(0)  
    }  
    var increment = { counter.value++ }  
    var decrement = { counter.value-- }
```

```
    Column(  
        modifier = modifier.fillMaxSize(),  
        //..  
    ) {  
        Button(onClick = { increment() }) {  
            Text(text = "Increment")  
        }  
        Text(text = "${counter.value}")  
        Button(onClick = { decrement() }) {  
            Text(text = "Decrement")  
        }  
    }  
}
```



Logic



View

Separate Logic and View

Separation of Concern

Create a ViewModel class for Logic

```
class MainScreenViewModel: ViewModel() {  
  
}
```

```
dependencies {  
  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose-android:2.9.3")  
    //...  
}
```

ViewModel

ViewModel keeps app data safe

- It stores data that does not get lost when the app's activity is destroyed and recreated.

ViewModel objects are durable

- Unlike activity instances, ViewModel objects are not destroyed.

Automatic data retention

- The app keeps ViewModel objects during configuration changes, so the data is ready right away after the app is rebuilt.

Expose read-only data from ViewModel

```
class MainScreenViewModel: ViewModel() {
```

```
    private var _counter = MutableStateFlow(0)
```

```
    val counter = _counter.asStateFlow()
```

```
    //...
```

```
}
```

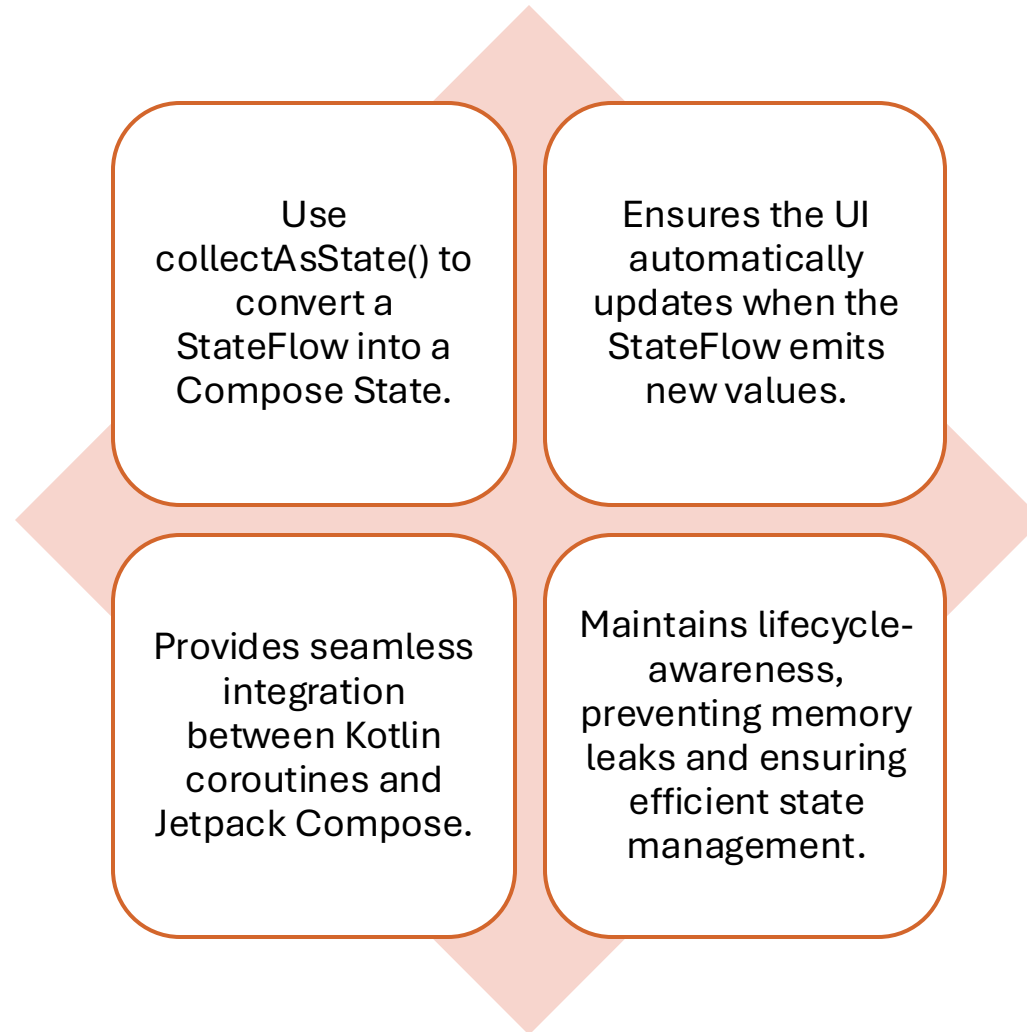
It is a type of StateFlow that allows its value to be updated.

- This converts the MutableStateFlow _counter to a read-only StateFlow using the asStateFlow() extension function.
- This means that while _counter can be updated within the MainActivityViewModel, external classes can only observe its value through counter and cannot modify it.

Include lambda(s) for increment & decrement

```
class MainScreenViewModel: ViewModel() {  
    private var _counter = MutableStateFlow(0)  
    val counter = _counter.asStateFlow()  
    var increment = {  
        _counter.value = _counter.value.plus(1)  
    }  
    var decrement = {  
        _counter.value = _counter.value.minus(1)  
    }  
}
```

Integrating StateFlow with Compose



Integrating StateFlow with Compose

```
@Composable
fun MainScreen(
    modifier: Modifier = Modifier,
    mainScreenViewModel : MainScreenViewModel = viewModel()
){

    val countState = mainScreenViewModel.counter.collectAsState()
    //...
}
```

Use `collectAsState()` to convert a `StateFlow` into a Compose State.

Ensures the UI automatically updates when the `StateFlow` emits new values.

Provides seamless integration between Kotlin coroutines and Jetpack Compose.

Maintains lifecycle-awareness, preventing memory leaks and ensuring efficient state management.

```
class MainActivityViewModel: ViewModel() {
    private var _counter = MutableStateFlow(value: 0)
    val counter = _counter.asStateFlow()
```

Integrating StateFlow with Compose

@Composable

```
fun MainScreen(  
    modifier: Modifier = Modifier,  
    mainScreenViewModel: MainScreenViewModel = viewModel()  
) {
```

1

```
    val countState = mainScreenViewModel.counter.collectAsState()
```

```
    Column(  
        //...
```

```
    ) {
```

```
        Button(onClick = {mainScreenViewModel.increment() }) {
```

```
            Text(text = "Increment")
```

```
        }
```

2

```
        Text(text = "${countState.value}")
```

```
        Button(onClick = {mainScreenViewModel.decrement() }) {
```

```
            Text(text = "Decrement")
```

```
        }
```

```
    }
```

```
}
```

UI State

- The UI is what the user sees, and the UI state is what the app says they should see.
- The UI is the visual representation of the UI state.
- Any changes to the UI state immediately are reflected in the UI.





UI State

- Keep UI State as immutable.
- Immutable objects guarantee that multiple sources do not alter the state of the app at an instant in time.
- This protection frees the UI to focus on a single role: reading state and updating UI elements accordingly.

Create a data class for UIState

```
data class CounterUIState(  
    val isLoading: Boolean = false,  
    val counter: Int = 0,  
    val errorMessage: String? = null  
)
```

This flag indicates whether the app is currently performing a task, like fetching data. It helps manage the UI, such as showing a loading spinner to the user.

This represents the current value of the counter. It's the main data point that the UI will display and update.

This optional string holds any error messages that might occur. If something goes wrong, this can be used to show an error message to the user.

Refactor the code in MainScreenViewModel

```
private var _counter = MutableStateFlow(0)
```



```
private var _counter = MutableStateFlow(CounterUIState(isLoading = false))
```

Refactor the code in MainScreenViewModel

```
var increment = {  
    viewModelScope.launch {  
        _counter.value = _counter.value.plus(1)  
    }  
}
```



```
var increment = {  
    viewModelScope.launch {  
        _counter.value.let {  
            _counter.value = it.copy(counter = it.counter.plus(1), errorMessage = null)  
        }  
    }  
}
```

Similarly, do it for decrement.

Refactor the code in MainScreenViewModel

```
var decrement = {  
    viewModelScope.launch {  
        _counter.value = _counter.value.minus(1)  
    }  
}
```



```
var decrement = {  
    viewModelScope.launch {  
        if (_counter.value.counter == 0) {  
            _counter.value = CounterUIState(errorMessage = "Counter cannot be less than 0")  
            return@launch  
        }  
        _counter.value.let {  
            _counter.value = it.copy(counter = it.counter.minus(1))  
        }  
    }  
}
```

Simulate loading in MainScreenViewModel

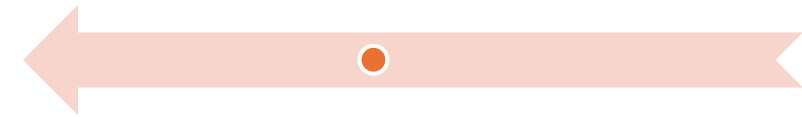
```
init {  
    viewModelScope.launch {  
        // Simulate loading state for 2 seconds  
        delay(2000L)  
        _counter.value = CounterUIState(isLoading = false, counter = 0)  
    }  
}
```

```

class MainScreenViewModel: ViewModel() {
    private var _counter = MutableStateFlow(CounterUIState(isLoading = true))
    val counter = _counter.asStateFlow()
    init {
        viewModelScope.launch {
            // Simulate loading state for 2 seconds
            delay(2000L)
            _counter.value = CounterUIState(isLoading = false, counter = 0)
        }
    }
    var increment = {
        viewModelScope.launch {
            _counter.value.let {
                _counter.value = it.copy(counter = it.counter.plus(1))
            }
        }
    }
    var decrement = {
        viewModelScope.launch {
            _counter.value.let {
                _counter.value = it.copy(counter = it.counter.minus(1))
            }
        }
    }
}

```

Refactored
MainScreenViewModel



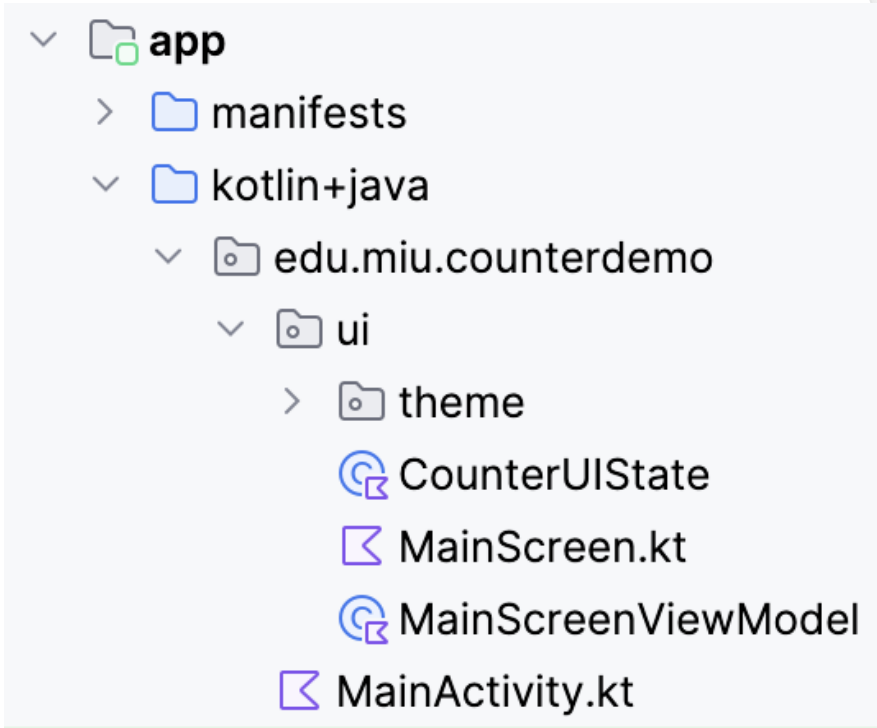
Refactor the code in MainScreen

```
Column(  
    //...  
) {  
    Button(onClick = { /*...*/ }) {  
        Text(text = "Increment")  
    }  
  
    Text(text = "${countState.value}")  
  
    Button(onClick = { /*...*/ }) {  
        Text(text = "Decrement")  
    }  
}
```

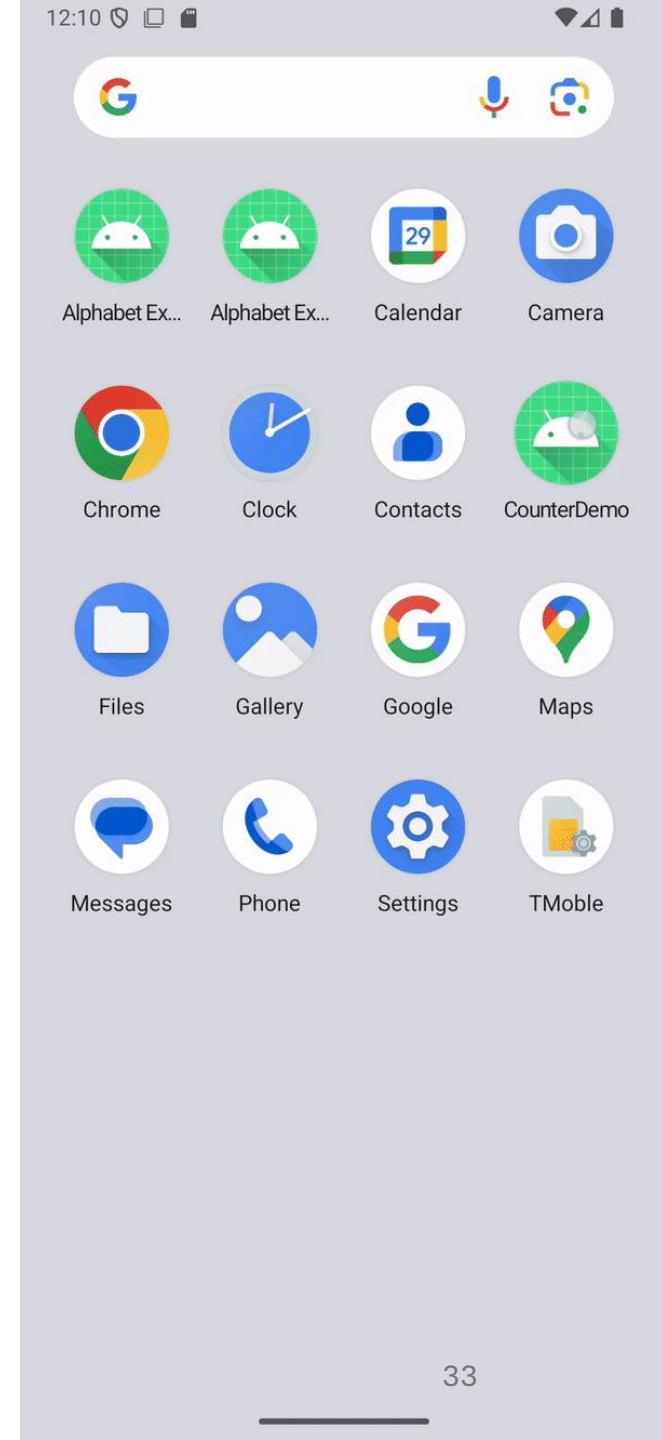
```
val state = countState.value  
when {  
    state.isLoading -> {  
        CircularProgressIndicator()  
    }  
    state.errorMessage != null -> {  
        Text(text = state.errorMessage)  
    }  
    else -> {  
        Text(text = "${state.counter}")  
    }  
}
```



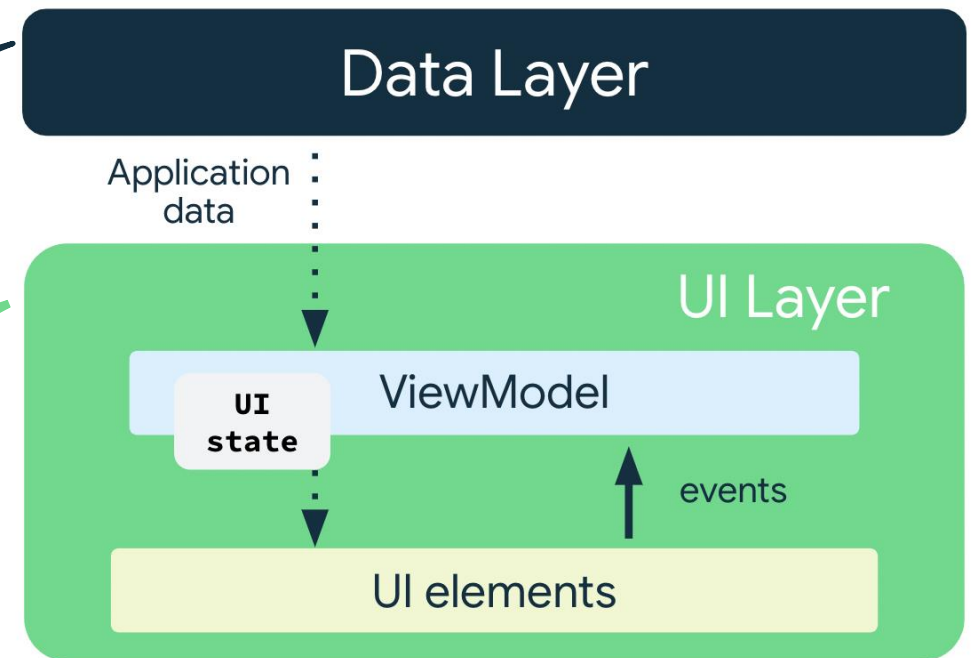
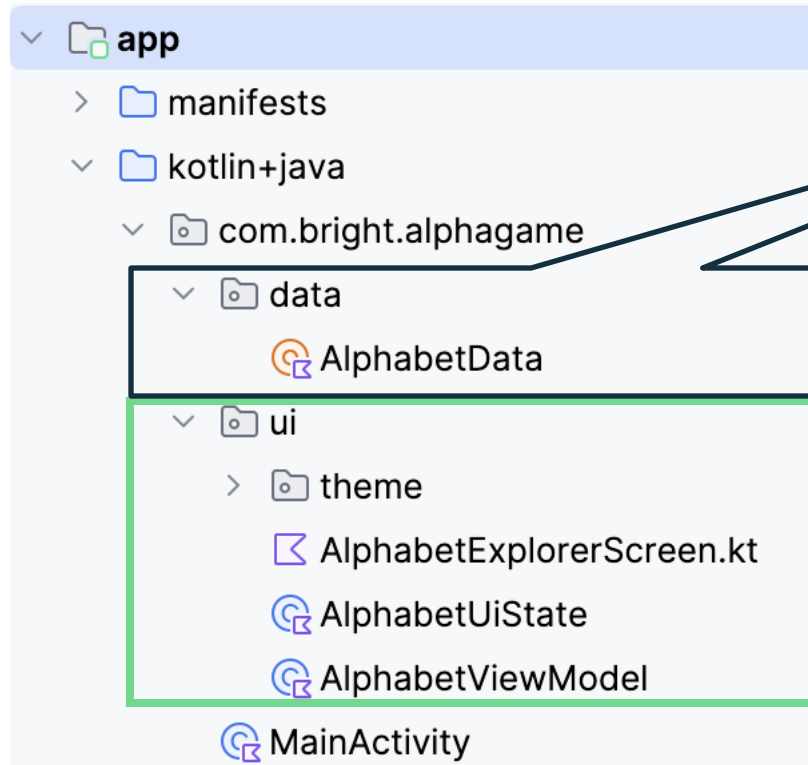
Package Hierarchy & Output



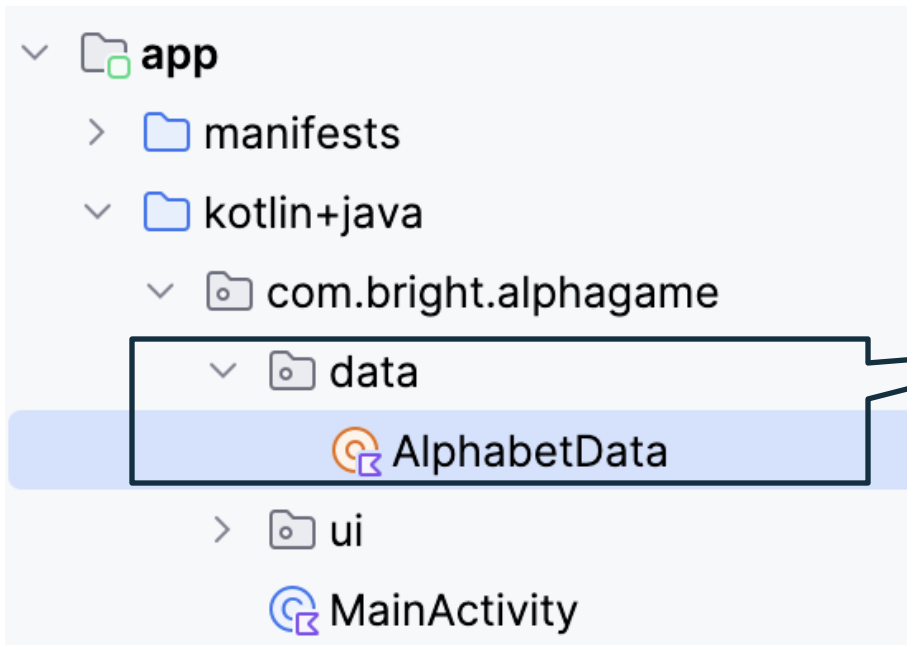
<https://github.com/brightgeevarghese/CounterDemo.git>



Create an alphabet explorer app which incorporates the recommended architecture

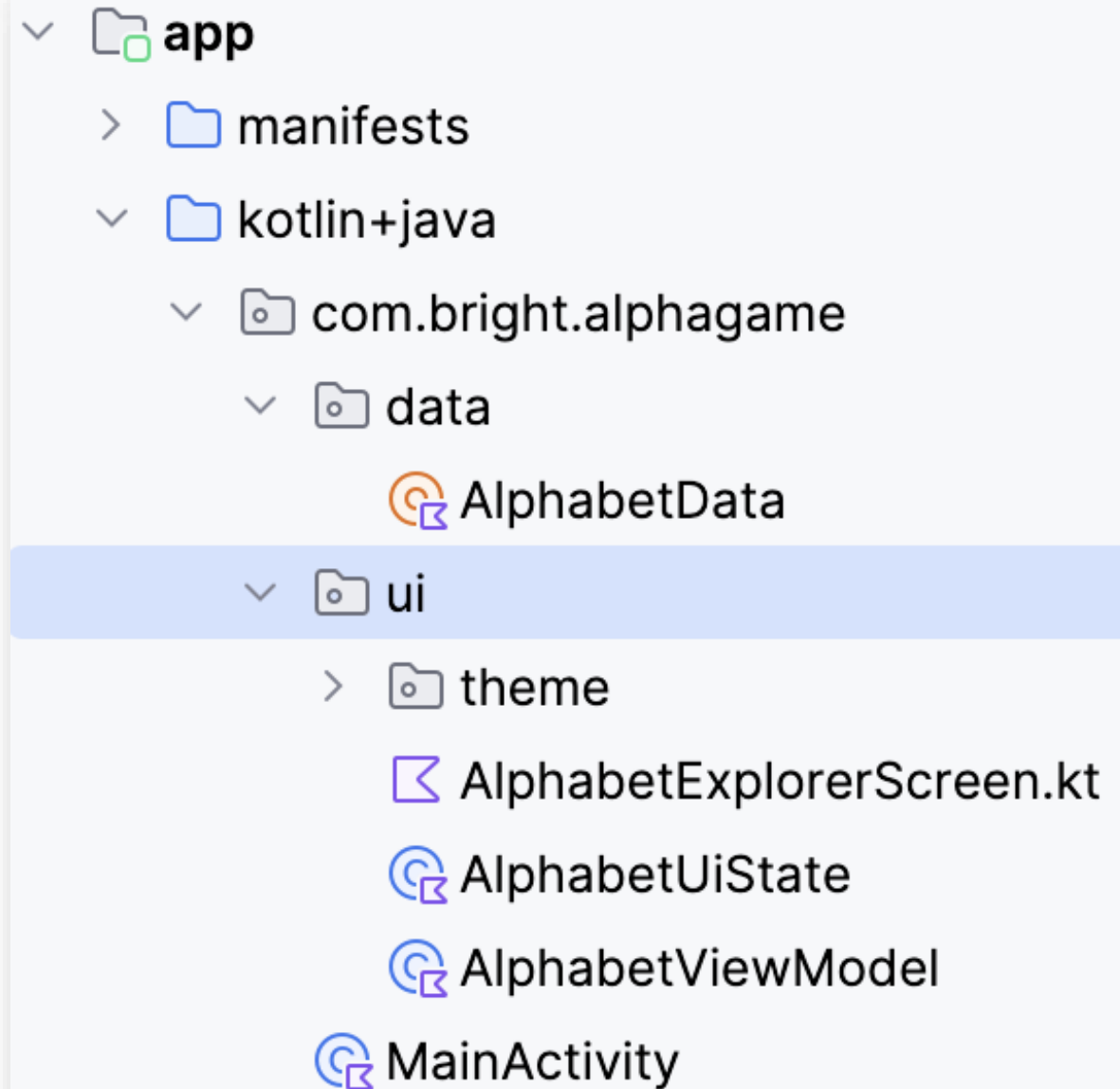


Create a data package to hold the data source

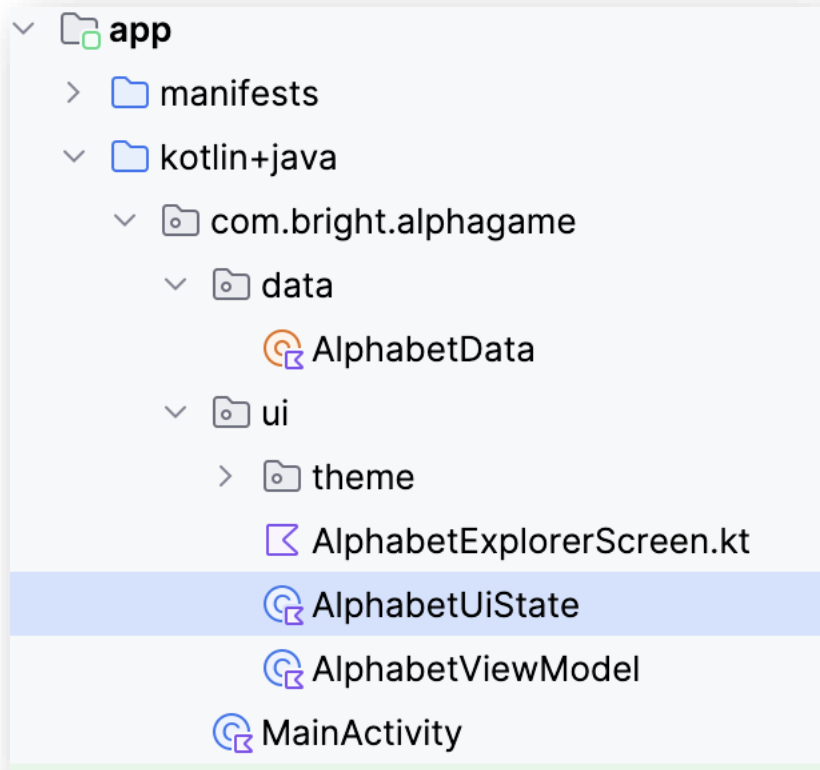


```
object AlphabetData{  
    val alphabets = listOf<Pair<Char, String>>(  
        'A' to "Alpha",  
        'B' to "Bravo",  
        'C' to "Charlie",  
        'D' to "Delta",  
        'E' to "Echo",  
        ...  
        'V' to "Victor",  
        'W' to "Whiskey",  
        'X' to "X-ray",  
        'Y' to "Yankee",  
        'Z' to "Zulu"  
    )  
}
```

Use the **ui** package to hold the ViewModel, UIState, and Composable function



Create a class for UIState



```
data class AlphabetUIState(  
    val currentAlphabet: Char,  
    val currentWord: String,  
    val isLoading: Boolean = false,  
    val isError: Boolean = false,  
    val isCompleted: Boolean = false  
)
```

Create ViewModel class

```
class AlphabetViewModel: ViewModel() {  
    // Retrieve the list of alphabet pairs from the data source  
    private val alphabets = AlphabetData.alphabets  
  
    private val _alphabetUIState = MutableStateFlow(  
        AlphabetUIState(  
            currentAlphabet = alphabets[0].first,  
            currentWord = alphabets[0].second,  
            isLoading = false,  
            isError = false,  
            isCompleted = true  
        )  
    )  
    // Expose _alphabetUIState as a read-only StateFlow to be observed by UI components  
    val alphabetUIState = _alphabetUIState.asStateFlow()  
  
    //...  
}
```

Include a
fun in
ViewModel
class to find
the next pair

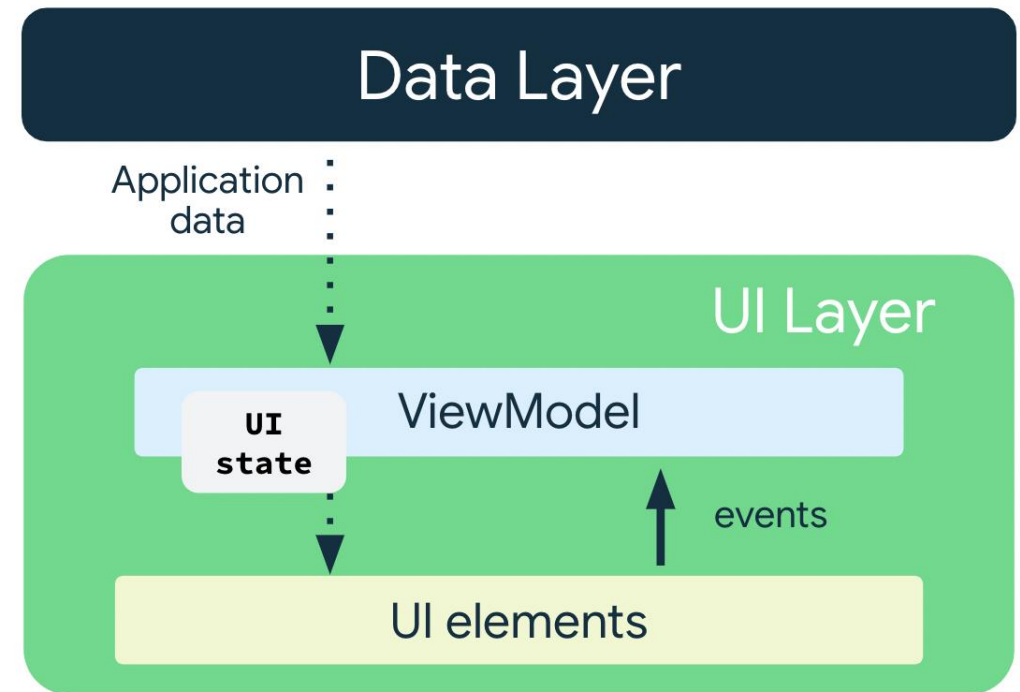
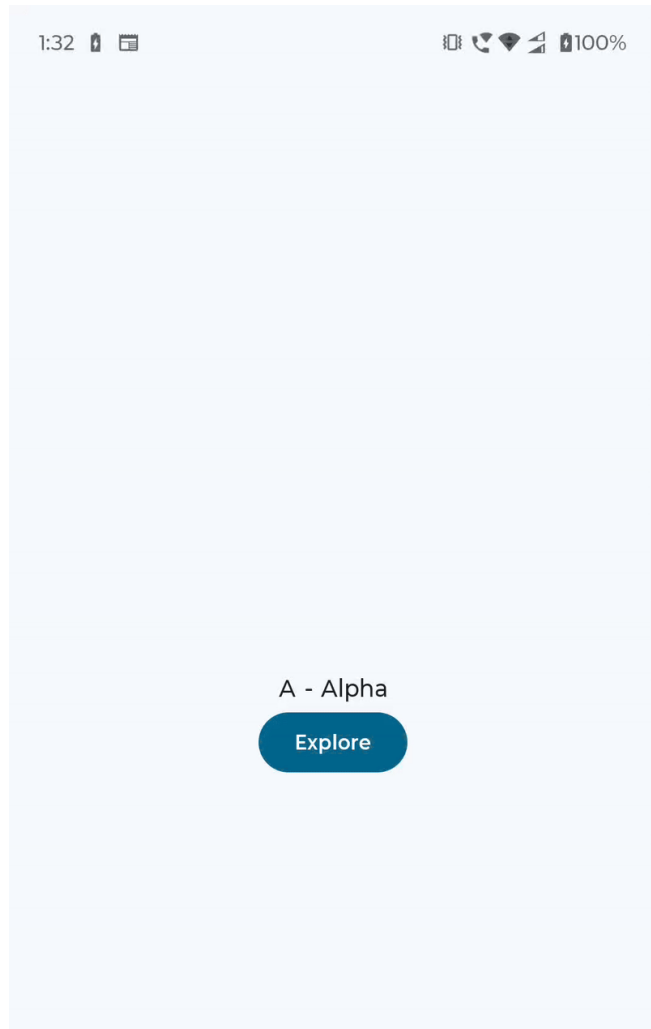
```
fun nextAlphabet() {  
    val currentIndex = alphabets.indexOf(  
        _alphabetUIState.value.currentAlphabet to _alphabetUIState.value.currentWord  
    )  
    if (currentIndex < alphabets.size - 1) {  
        val nextAlphabet = alphabets[currentIndex + 1]  
        _alphabetUIState.value = _alphabetUIState.value.copy(  
            currentAlphabet = nextAlphabet.first,  
            currentWord = nextAlphabet.second,  
            isLoading = false,  
            isError = false,  
            isCompleted = true  
        )  
    } else if (currentIndex == alphabets.size - 1) {  
        _alphabetUIState.value = _alphabetUIState.value.copy(  
            currentAlphabet = alphabets[0].first,  
            currentWord = alphabets[0].second  
        )  
    }  
}
```

Create a
composable fun

```
@Composable
fun AlphabetExplorerScreen(
    modifier: Modifier = Modifier,
    alphabetViewModel: AlphabetViewModel = viewModel()
){
    val alphabetUIState by alphabetViewModel.alphabetUIState.collectAsState()
    Column( /*...*/ ) {
        val alphabet = alphabetUIState.currentAlphabet
        val word = alphabetUIState.currentWord
        when {
            alphabetUIState.isLoading -> { Text(text = "Loading...") }
            alphabetUIState.isError -> { Text(text = "Error loading alphabets") }
            alphabetUIState.isCompleted -> {
                Text(text = "$alphabet for $word")
            }
        }
        Button(onClick = { alphabetViewModel.nextAlphabet() }) {
            Text(text = "Next Alphabet")
        }
    }
}
```

UI elements + UI State + ViewModel + Data

<https://github.com/brightgeevarghese/AlphabetExplorer.git>





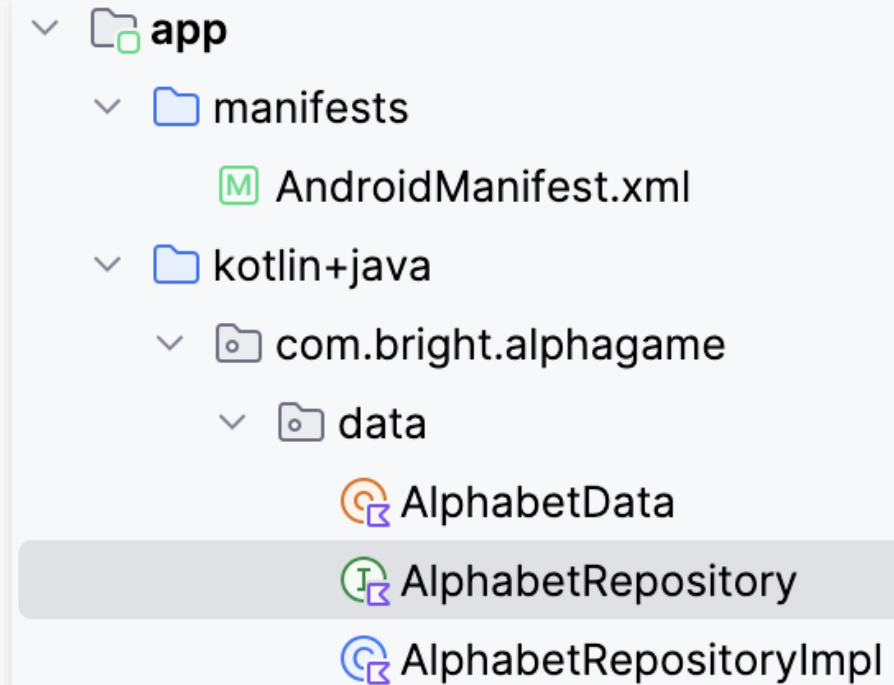
Repository Pattern



Repository Pattern

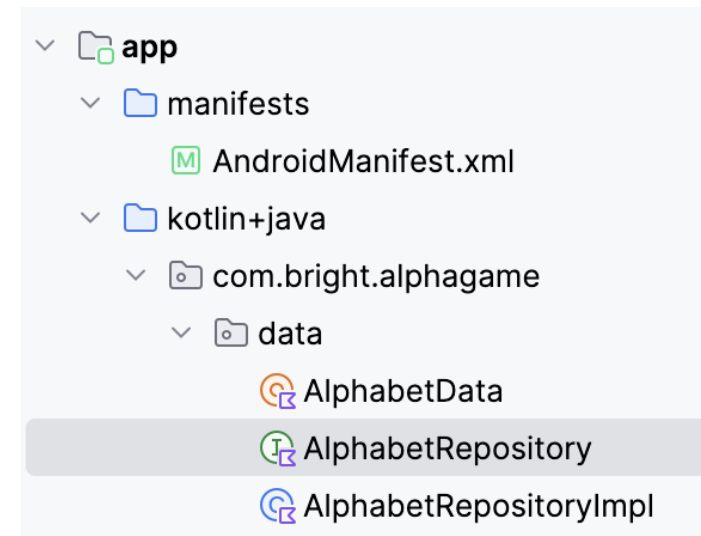
- Without Repository
 - The ViewModel directly accesses the database or API.
 - Leads to tight coupling and duplicate data-handling code.
 - Difficult to test or replace data sources later.
- With Repository
 - The ViewModel depends only on the Repository interface.
 - Repository decides where data comes from (local DB, remote API, or both).
 - Promotes Clean Architecture and single responsibility.

Create a Repository



```
interface AlphabetRepository {  
    fun getAlphabets(): List<Pair<Char, String>>  
    fun getNextAlphabet(currentAlphabet: Pair<Char, String>): Pair<Char, String>  
}
```

Implement the Repository

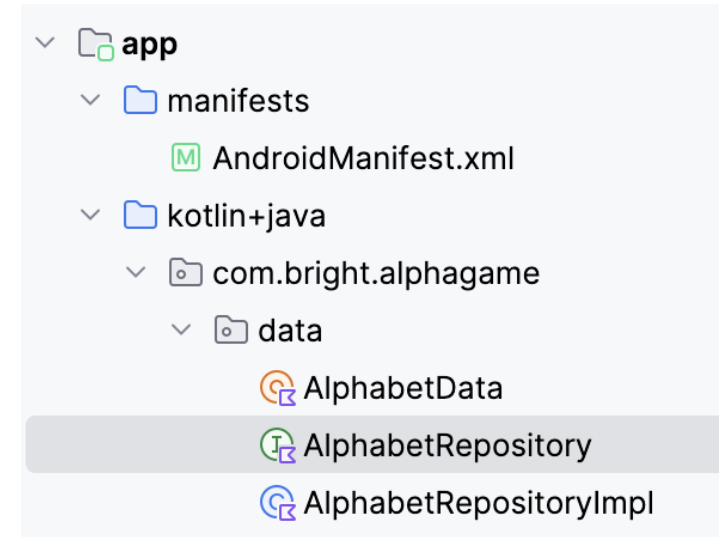


```
class AlphabetRepositoryImpl(  
    private val alphabets: List<Pair<Char, String>> = AlphabetData.alphabets  
) : AlphabetRepository {...}
```

Implement the Repository

```
override fun getAlphabets(): List<Pair<Char, String>> = alphabets
```

```
override fun getNextAlphabet(currentAlphabet: Pair<Char, String>): Pair<Char, String> {  
    val currentIndex = AlphabetData.alphabets.indexOf(currentAlphabet)  
    if (currentIndex < AlphabetData.alphabets.size - 1) {  
        return AlphabetData.alphabets[currentIndex + 1]  
    } else {  
        return AlphabetData.alphabets.first()  
    }  
}
```



Include repository as a constructor parameter

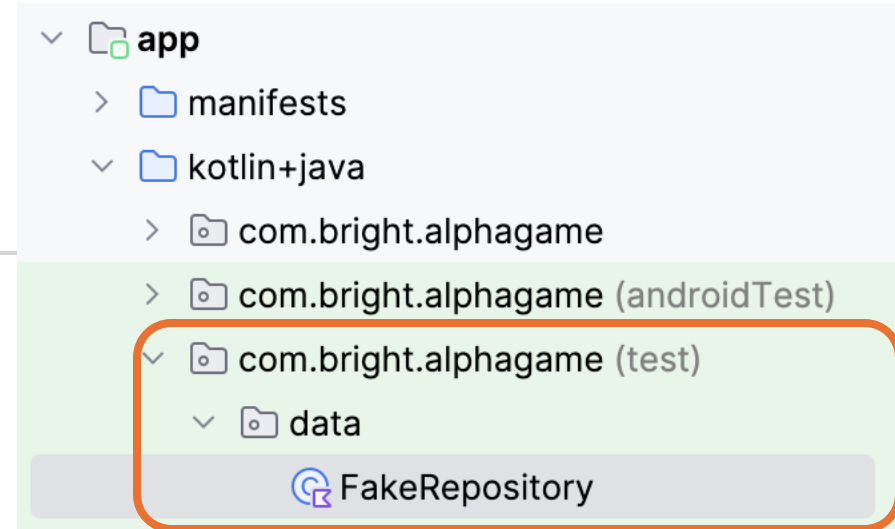
```
class AlphabetViewModel (  
    val alphabetRepository: AlphabetRepository  
) : ViewModel() {  
    // private val alphabets = AlphabetData.alphabets  
    private val alphabets = alphabetRepository.getAlphabets()  
    //...  
}
```

Unit Testing

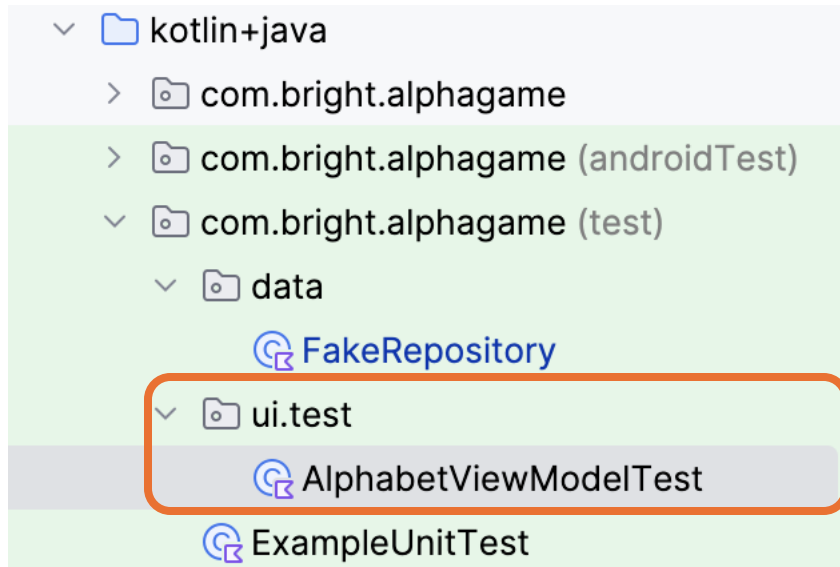
To test the ViewModel

Set up a Fake Repository

```
class FakeRepository(  
    alphabets: List<Pair<Char, String>> = listOf(  
        'A' to "Alpha",  
        'B' to "Bravo",  
        'C' to "Charlie"  
    )  
): AlphabetRepository by AlphabetRepositoryImpl(alphabets)  
  
// The class delegates all functionality to AlphabetRepositoryImpl  
// using Kotlin's delegation feature (the by keyword).
```



Create a Test class, **AlphabetViewModelTest**



```
class AlphabetViewModelTest {  
    private lateinit var viewModel: AlphabetViewModel  
  
    @Before  
    fun setup() {  
        viewModel = AlphabetViewModel(FakeRepository())  
    }  
    // ...  
}
```

Write a test to check initial state

@Test

```
fun testInitialState() {  
    val uiState = viewModel.alphabetUiState.value  
    assert(uiState.currentChar == 'A')  
    assert(uiState.currentWord == "Alpha")  
}
```

Write a test to check the next alphabet after initial state

@Test

```
fun testNextAlphabet() {  
    viewModel.getNextAlphabet()  
    val uiState = viewModel.alphabetUiState.value  
    assert(uiState.currentChar == 'B')  
    assert(uiState.currentWord == "Bravo")  
}
```

Write a test to check the completed alphabet

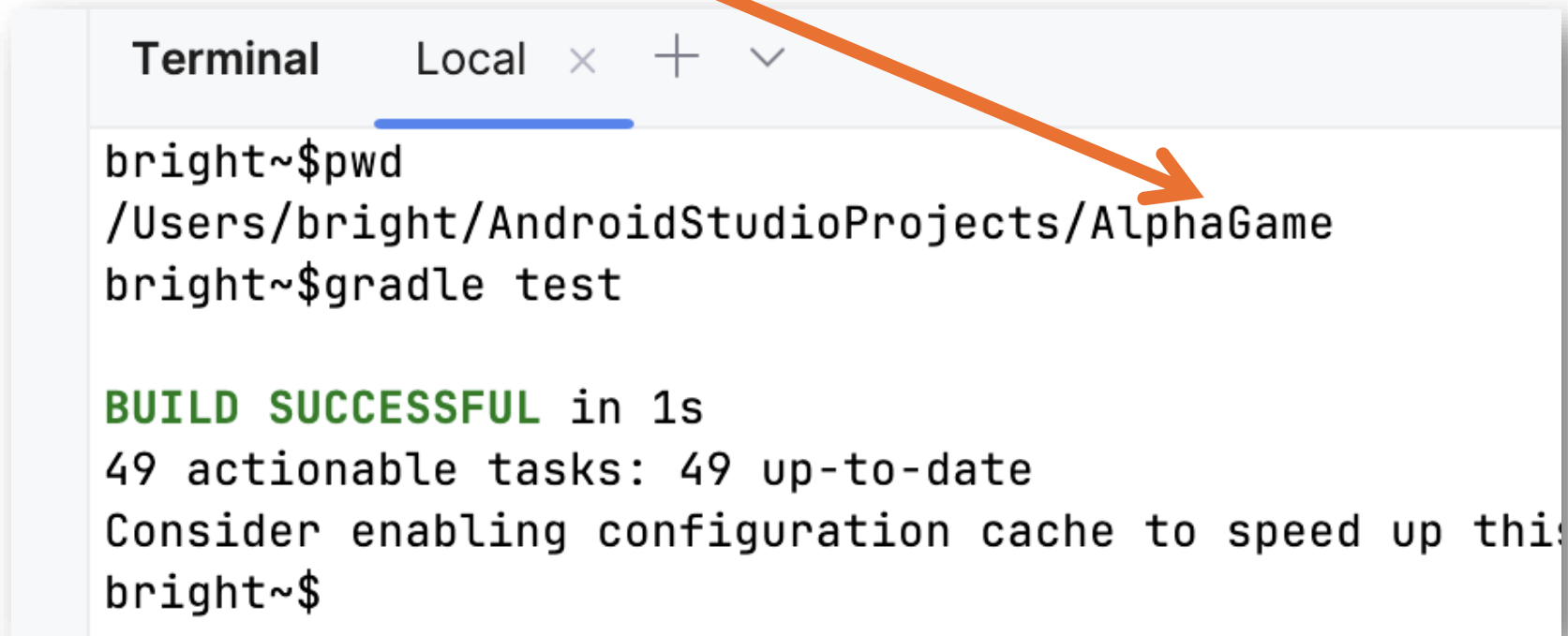
@Test

```
fun testCompletedAlphabet() {  
    repeat(2) {  
        viewModel.getNextAlphabet()  
    }  
    val uiState = viewModel.alphabetUiState.value  
    assert(uiState.isCompleted)  
}
```

```
class FakeRepository(  
    alphabets: List<Pair<Char, String>> = listOf(  
        'A' to "Alpha",  
        'B' to "Bravo",  
        'C' to "Charlie"  
    )  
) : AlphabetRepository by AlphabetRepositoryImpl(alphabets)
```

Run all test cases

Go to the project folder



The image shows a terminal window with a tab labeled 'Terminal' and a sub-tab 'Local'. An orange arrow points from the text 'Go to the project folder' to the terminal output. The terminal shows the following commands and output:

```
bright~$pwd
/Users/bright/AndroidStudioProjects/AlphaGame
bright~$gradle test

BUILD SUCCESSFUL in 1s
49 actionable tasks: 49 up-to-date
Consider enabling configuration cache to speed up this build
bright~$
```

Class com.bright.alphagame.ui.test.AlphabetViewModelTest

all > com.bright.alphagame.ui.test > AlphabetViewModelTest

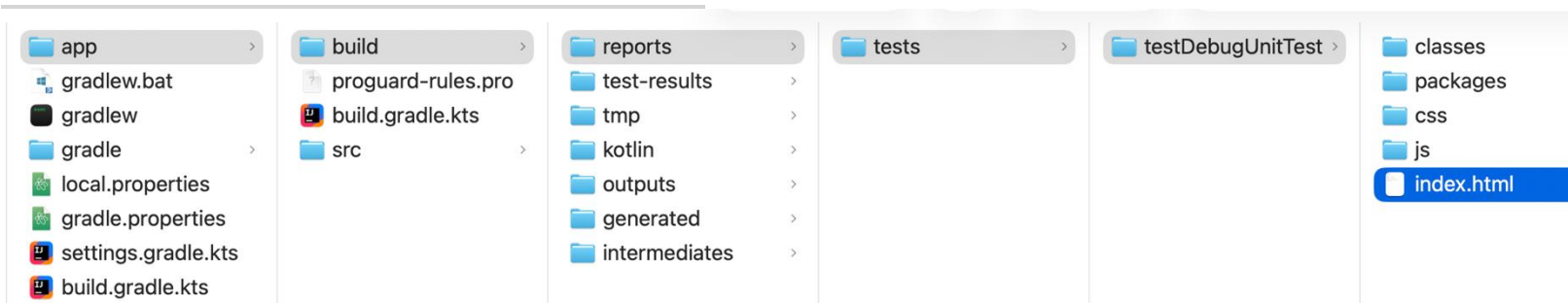
3 tests
0 failures
0 ignored
0.030s duration

100%
successful

Examine the test report

Tests

Test	Duration	Result
testCompletedAlphabet	0s	passed
testInitialUiState	0.030s	passed
testNextAlphabet	0s	passed



Dependency injection with Hilt

goo.gle/hilt-cheatsheet

Hilt

Definition

- Hilt is a dependency injection (DI) library for Android.
- Built on top of Dagger, designed specifically for Android to reduce boilerplate.
- Hilt is Jetpack's recommended DI.

Why Use Hilt?

- Simplifies the process of dependency injection.
- Provides a standard way to handle dependencies in Android apps.

Setting Up Hilt

- Add the hilt-android-gradle-plugin plugin to your project's root build.gradle file:

// Top-level build file where you can add configuration options common to all sub-projects/modules.

```
plugins {  
    alias(libs.plugins.android.application) apply false  
    alias(libs.plugins.kotlin.android) apply false  
    alias(libs.plugins.kotlin.compose) apply false  
  
    // Hilt plugin for Dependency Injection  
    // but not applied globally (apply false).  
    // Keeps it available for modules that explicitly need it.  
    id("com.google.dagger.hilt.android") version "2.57.2" apply false  
    // Kotlin Symbol Processing (KSP) plugin  
    // Applied immediately at the root with version "2.2.10-2.0.2".  
    // Required to run annotation processors (e.g., Room, Moshi, Hilt codegen).  
    id("com.google.devtools.ksp") version "2.2.10-2.0.2" apply false  
}
```

Setting Up Hilt

Apply the Gradle plugin and add these dependencies in your app/build.gradle file:

```
import org.jetbrains.kotlin.gradle.dsl.JvmTarget

plugins {
    // ...
    id("com.google.devtools.ksp")
    id("com.google.dagger.hilt.android")
}

android {
    // kotlinOptions {
    // jvmTarget = "11"
    // }
    kotlin {
        compilerOptions {
            jvmTarget.set(JvmTarget.JVM_11)
        }
    }
}

dependencies {
    implementation("com.google.dagger:hilt-android:2.57.2")
    //dependency for hiltViewModel()
    implementation("androidx.hilt:hilt-navigation-compose:1.3.0")
    ksp("com.google.dagger:hilt-android-compiler:2.57.2")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.9.4")
    // ...
}
```

Create an Application Class

`@HiltAndroidApp`
`class MyApp: Application()`

- All apps that use Hilt must contain an Application class, annotated with `@HiltAndroidApp`.
- It triggers Hilt's code generation at compile time.

Register Application class in AndroidManifest.xml

```
AndroidManifest.xml x
1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:tools="http://schemas.android.com/tools">
4
5      <application
6          android:name=".MyApp"
7          android:allowBackup="true"
8          android:dataExtractionRules="@xml/data_extraction_rules"
9          android:fullBackupContent="@xml/backup_rules"
```

@HiltAndroidApp
class MyApp: Application()

If you miss this step:

Caused by: java.lang.IllegalStateException: Hilt Activity must be attached to an @HiltAndroidApp Application.
Did you forget to specify your Application's class name in your manifest's <application />'s android:name attribute?

Create an Application Class

`@HiltAndroidApp`
`class MiuApp: Application()`

- All apps that use Hilt must contain an Application class, annotated with `@HiltAndroidApp`.
- It triggers Hilt's code generation at compile time.

Quick Start

*// @AndroidEntryPoint tells Hilt that this Activity is a dependency injection entry point.
// Hilt will generate code so that this Activity (and any Composables/ViewModels it uses)
// can receive dependencies (like repositories, use cases, etc.) from Hilt's graph.*

@AndroidEntryPoint

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            AlphaGameTheme {  
                AlphabetExplorerScreen()  
            }  
        }  
    }  
}
```



Create a Hilt module that provides a dependency for `DataRepository`

A Hilt module is a class that provides dependencies to the Hilt dependency injection framework.

It tells Hilt how to create instances of certain types, which can then be injected into other parts of your app.

Create a Hilt module that provides a dependency for Repository

// Marks this class as a Hilt module.

// A module is a place where you define how to provide dependencies.

@Module

// Tells Hilt in which component this module will be installed.

// SingletonComponent means the provided dependencies live as long as the Application.

@InstallIn(SingletonComponent::class)

object RepositoryModule {

@Provides

fun provideAlphabetRepository(): AlphabetRepository {

return AlphabetRepositoryImpl()

 }

}

Dependency Injection for ViewModel by Hilt

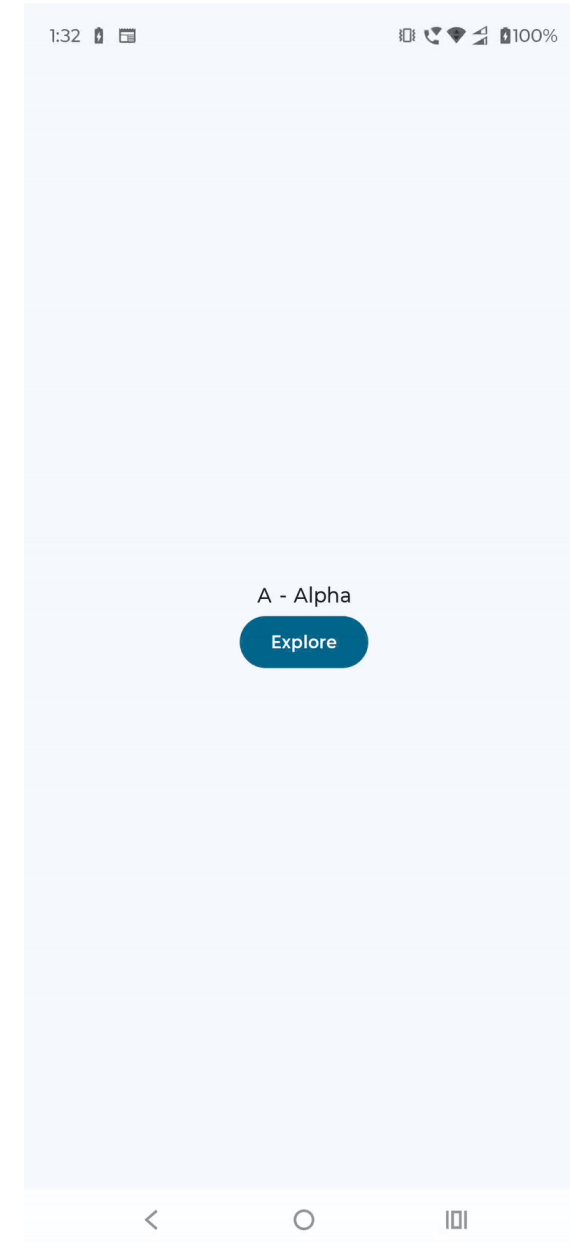
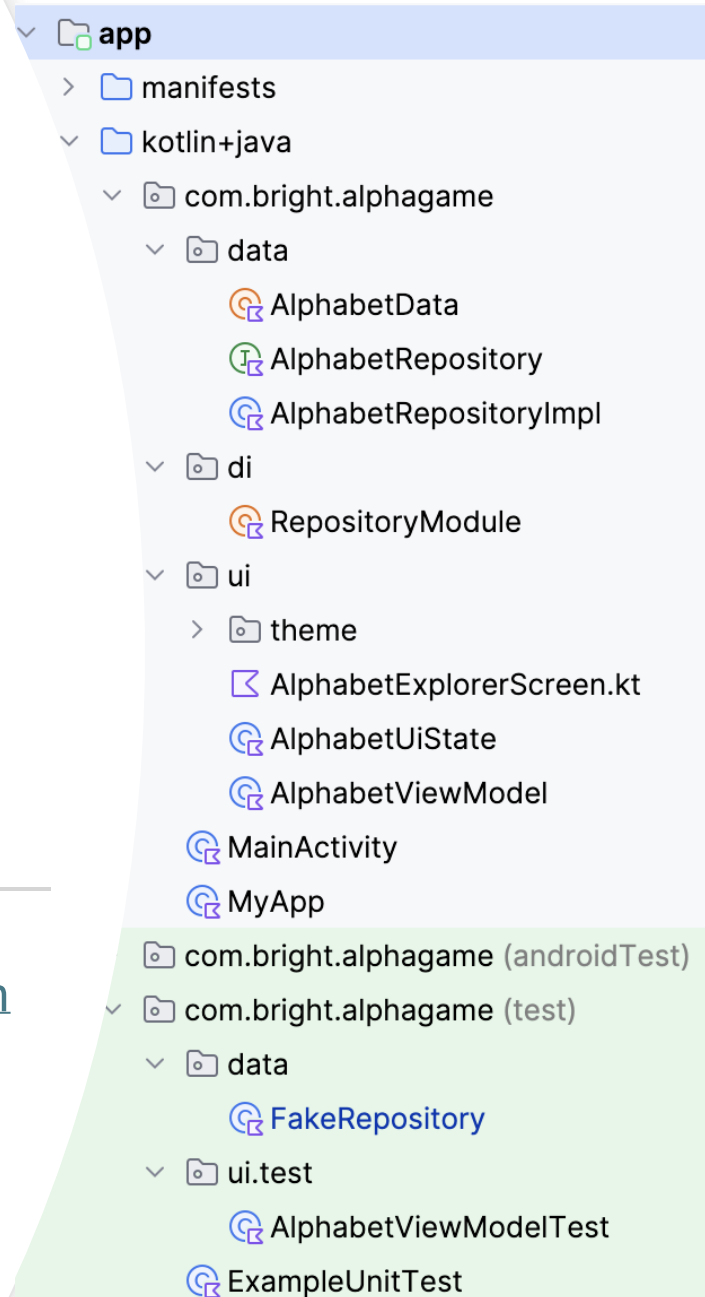
- @HiltViewModel
 - Tells Hilt how to provide instances of an Architecture Component ViewModel.

```
@HiltViewModel
class AlphabetViewModel @Inject constructor(
    val alphabetRepository: AlphabetRepository
): ViewModel() {...}
```

```
val alphabetViewModel: AlphabetViewModel = hiltViewModel()
```

Hilt + ViewModel

<https://github.com/brightgeevarghese/AlphabetExp.git>



Another Example using Hilt

Quick Start

1 @AndroidEntryPoint

```
class MainActivity : ComponentActivity() {
```

2 @Inject lateinit var musicPlayer: MediaPlayer

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        musicPlayer.playMusic()
```

```
    }
```

```
}
```

1

- It enables Hilt to inject dependencies into an activity.
- All Hilt-injected variables will be available when `super.onCreate()` is called.
- Hilt injects `MediaPlayer` in the Activity.

2

- Hilt injects `MediaPlayer` in the Activity.

How does Hilt know to provide an instance of MusicPlayer?

- Annotate the constructor of the class, MusicPlayer with @Inject

```
class MusicPlayer @Inject constructor() {  
    fun playMusic() { /*...*/ }  
}
```

Manual DI

- When an activity is annotated with **@AndroidEntryPoint**, a dependency container, say MainActivityContainer is automatically created, managed, and associated with that Activity.

//MainActivity annotated with @AndroidEntryPoint

```
class MainActivityContainer {
```

//MediaPlayer annotated with @Inject

```
    fun provideMediaPlayer() = MediaPlayer()
```

```
}
```

Manual DI

```
class MainActivity : ComponentActivity() {  
    private lateinit var container: MainActivityContainer  
    private lateinit var musicPlayer: MusicPlayer  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // AndroidEntryPoint does this for you  
        container = MainActivityContainer()  
        musicPlayer = container.provideMusicPlayer()  
        super.onCreate(savedInstanceState)  
        musicPlayer.playMusic()  
    }  
}
```

Annotations Recap

Ref:

<https://developer.android.com/images/training/dependency-injection/hilt-annotations.pdf>

@HiltAndroidApp

Kicks off Hilt code generation.
Must annotate the Application class.

```
@HiltAndroidApp  
class MyApplication : Application() { ... }
```

@AndroidEntryPoint

Adds a DI container to the Android
class annotated with it.
This requires using Hilt's Gradle Plugin.

```
@AndroidEntryPoint  
class MainActivity : AppCompatActivity() { ... }
```

@Inject

Constructor Injection. Tells which
constructor to use to provide instances
and which dependencies the type has.

```
class AnalyticsAdapter @Inject constructor(  
    private val service: AnalyticsService  
) { ... }
```

Field injection. Populates fields in
@AndroidEntryPoint annotated classes.
Fields cannot be private.

```
@AndroidEntryPoint  
class MainActivity : AppCompatActivity() {  
    @Inject lateinit var adapter: AnalyticsAdapter  
    ...  
}
```

Inject other types

- Interfaces do not have constructors, and therefore, we cannot use the `@Inject` annotation directly on an interface's constructor.
- Instead, provide dependencies that implement the interface using Hilt's binding methods.

```
class MusicPlayer @Inject constructor(  
    private val repository: MusicRepository  
) {  
    fun playMusic() { /*...*/ }  
}
```

```
interface MusicRepository { /*...*/ }
```

Manual DI

```
class MainActivityContainer {  
    fun provideMusicRepository(): MusicRepository {  
        return MusicRepositoryImpl()  
    }  
    fun provideMusicPlayer() = MusicPlayer(  
        provideMusicRepository()  
    )  
}
```



Create a Hilt module that provides a dependency for `DataRepository`

A Hilt module is a class that provides dependencies to the Hilt dependency injection framework.

It tells Hilt how to create instances of certain types, which can then be injected into other parts of your app.

Create a Hilt module that provides a dependency for `DataRepository`

```
1 @Module
2 @InstallIn(SingletonComponent::class)
3
object ProgramRepositoryModule {
    @Provides
    fun provideProgramRepository(): ProgramRepository {
        return ProgramRepositoryImpl()
    }
}
```

1. **@Module** annotation marks the class as a Hilt module. A module tells Hilt how to provide instances of certain types.
2. **@InstallIn(SingletonComponent::class)** specifies which Hilt component this module will be installed in
3. **SingletonComponent** ensures that dependencies provided by this module have a singleton scope, meaning they are created once and shared across the entire app.
4. Declaring the module as an object makes it a singleton.
5. This ensures that the module itself is instantiated only once, following Kotlin's singleton pattern.⁷⁷

Create a Hilt module that provides a dependency for DataRepository

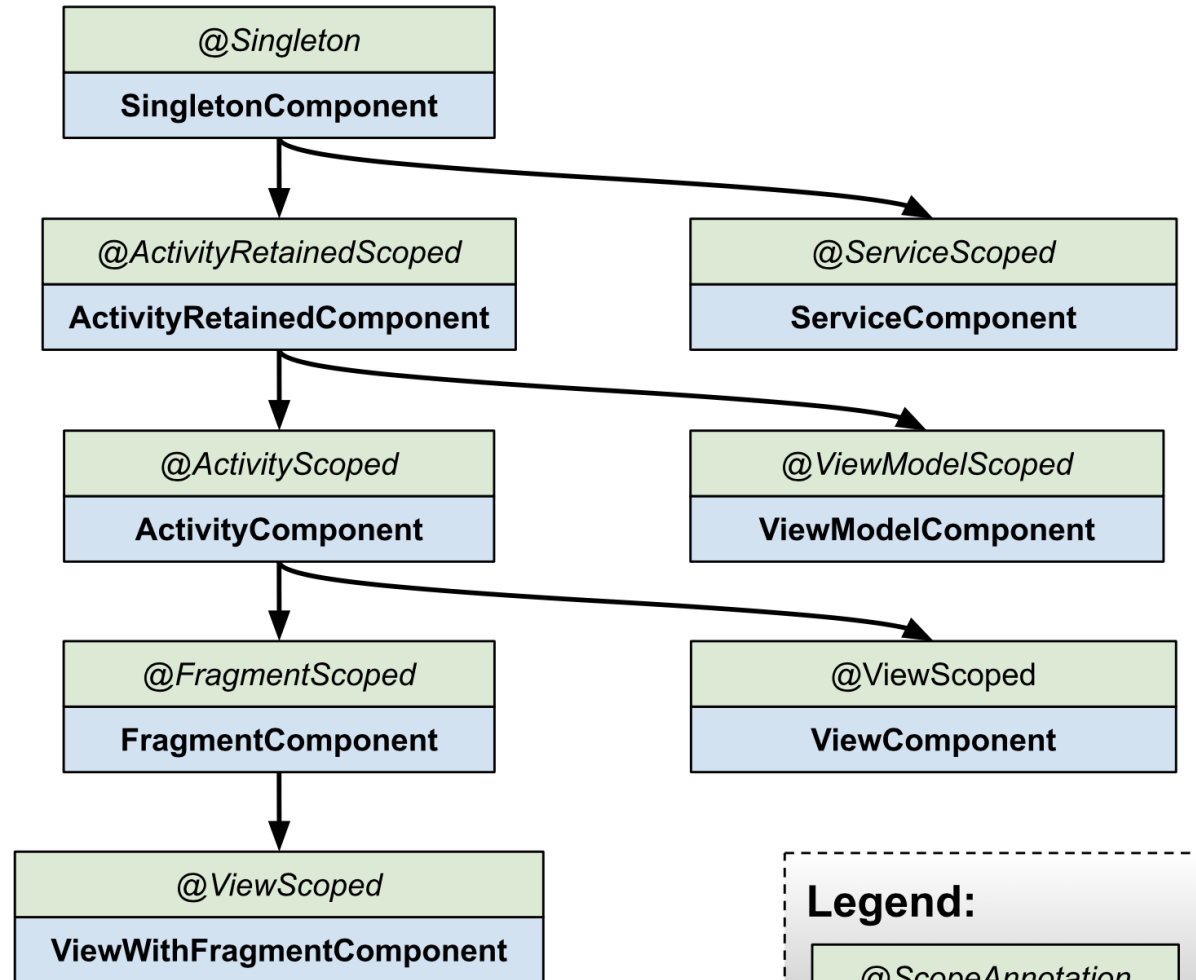
```
@Module
@InstallIn(SingletonComponent::class)
object ProgramRepositoryModule {

    @Provides
    fun provideProgramRepository(): ProgramRepository {
        return ProgramRepositoryImpl()
    }
}
```

1. **@Provides** annotation tells Hilt that the following method will provide a specific dependency.
 1. Hilt will call this method to obtain an instance of the type specified.
2. **provideProgramRepository()** is a function that Hilt uses to create and provide an instance of ProgramRepository.

Hierarchy of the components that Hilt generates

<https://developer.android.com/training/dependency-injection/hilt-android>



Legend:

@ScopeAnnotation

ComponentName

Note:

- When you use `@InstallIn(SingletonComponent::class)` in a Hilt module, the dependencies provided by that module are available to components that are subcomponents of the `SingletonComponent`.
- This is the top-level component in Hilt, with a lifecycle tied to the application's lifecycle (it lives as long as the application does).

Annotations Recap

Ref: <https://developer.android.com/images/training/dependency-injection/hilt-annotations.pdf>

@Module	Class in which you can add bindings for types that cannot be constructor injected.	<pre>@InstallIn(SingletonComponent::class) @Module class AnalyticsModule { ... }</pre>
@InstallIn	Indicates in which Hilt-generated DI containers (<code>SingletonComponent</code> in the code) module bindings must be available.	<pre>@InstallIn(SingletonComponent::class) @Module class AnalyticsModule { ... }</pre>
@Provides	Adds a binding for a type that cannot be constructor injected: - Return type is the binding type. - Parameters are dependencies. - Every time an instance is needed, the function body is executed if the type is not scoped.	<pre>@InstallIn(SingletonComponent::class) @Module class AnalyticsModule { @Provides fun providesAnalyticsService(converterFactory: GsonConverterFactory): AnalyticsService { return Retrofit.Builder() .baseUrl("https://example.com") .addConverterFactory(converterFactory) .build() .create(AnalyticsService::class.java) } }</pre>

Dependency Injection for ViewModel by Hilt

- @HiltViewModel
 - Tells Hilt how to provide instances of an Architecture Component ViewModel.

```
@HiltViewModel
```

```
class SharedViewModel @Inject constructor(
```

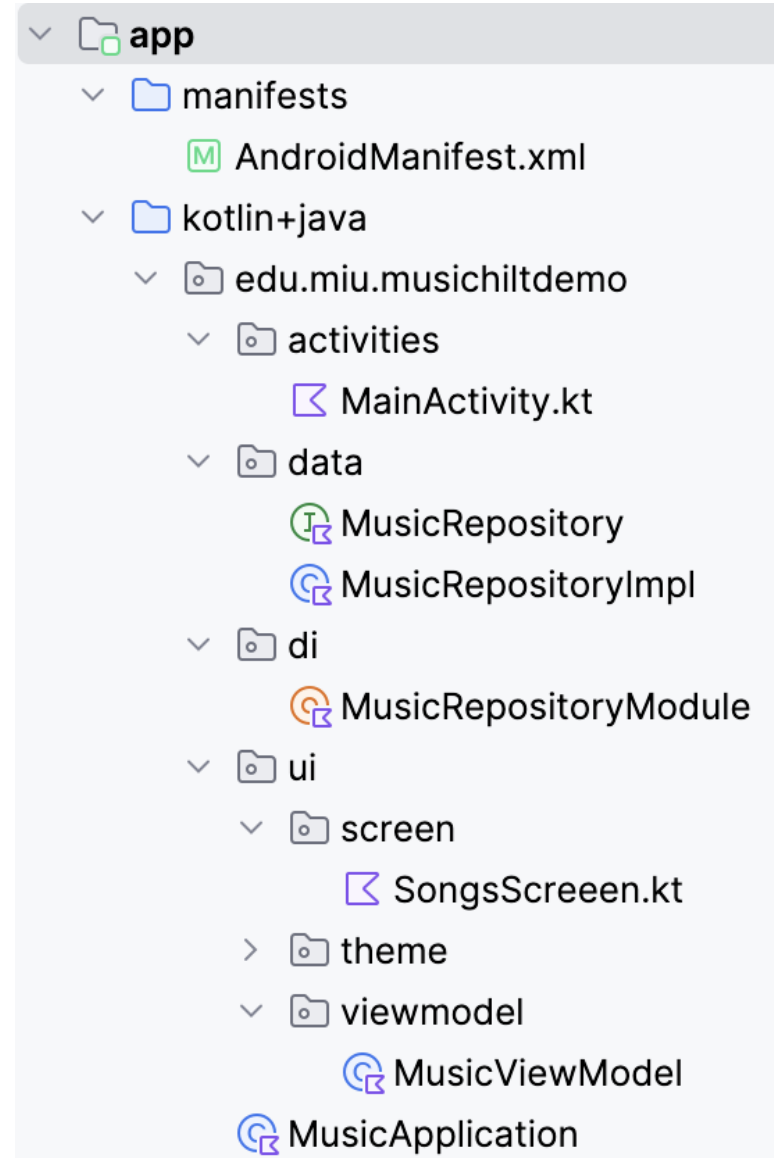
```
    private val repository: MusicRepository
```

```
) : ViewModel() { /*...*/ }
```

```
val sharedViewModel: SharedViewModel = hiltViewModel()
```

Hilt + ViewModel

<https://github.com/brightgeevarghese/MusicHiltDemo.git>





Radio Button



RadioButton

@Composable

fun RadioButton(

selected: Boolean,

onClick: (() -> Unit)?,

modifier: Modifier = Modifier,

enabled: Boolean = true,

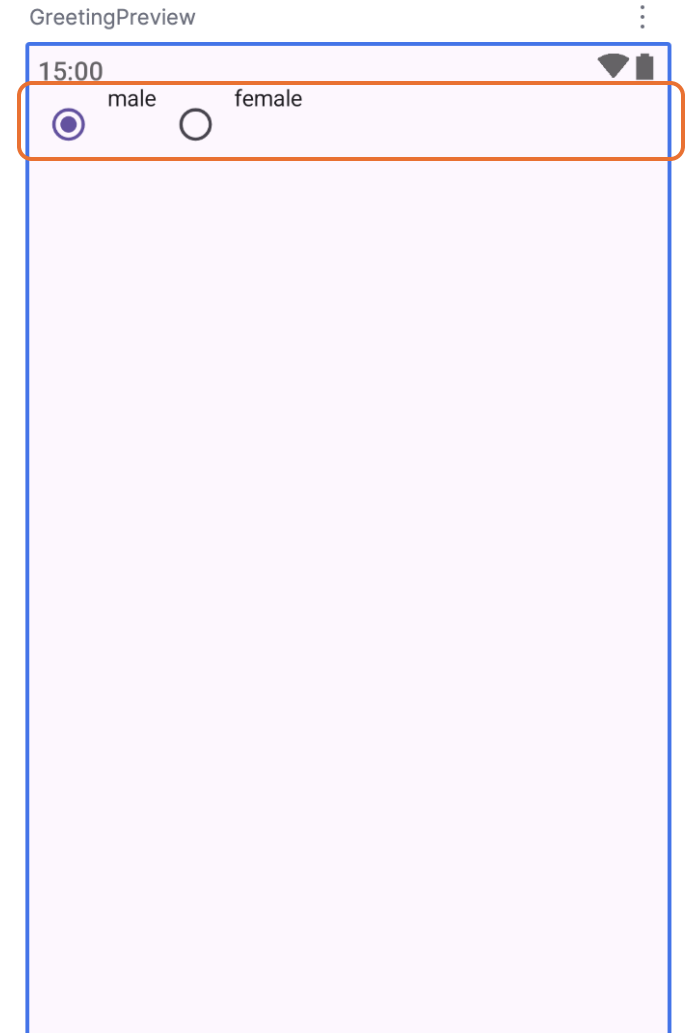
colors: RadioButtonColors = RadioButtonDefaults.colors(),

interactionSource: MutableInteractionSource? = null

)

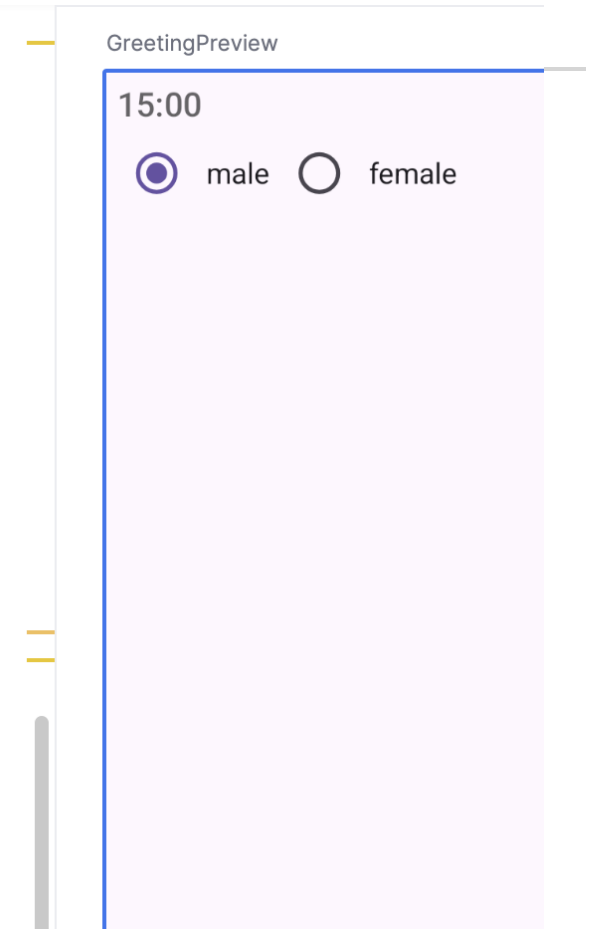
Create two RadioButton

```
val options = listOf("male", "female")
var selectedOption by remember { mutableStateOf(value = options[0]) }
Column(modifier = modifier) {
    Column {
        Row {
            options.forEach { option->
                RadioButton(
                    selected = (option == selectedOption),
                    onClick = { selectedOption = option }
                )
                Text(
                    text = "$option"
                )
            }
        }
    }
}
```



Align Center Vertically

```
Column(modifier = modifier) {  
    Column {  
        Row(  
            verticalAlignment = Alignment.CenterVertically  
        ) {  
            options.forEach {option->  
                RadioButton(...)  
                Text(  
                    text = "$option"  
                )  
            }  
        }  
    }  
}
```



Add a Button

```
val context = LocalContext.current
val options = listOf("male", "female")
var selectedOption by remember { mutableStateOf(value = options[0]) }

Column(modifier = modifier) {
    Column {
        Row(...) { ... }
    }
    Button(
        onClick = {
            Toast.makeText(
                context,
                text = "Selected option: $selectedOption",
                duration = Toast.LENGTH_SHORT).show()
        },
        modifier = Modifier.align(Alignment.Start)
    ) {
        Text(text = "Submit")
    }
}
```

15:00

☒ male ☐ female

Submit

```
Column {
    Row(...) {
        options.forEach { option->
            RadioButton(
                selected = (option == selectedOption),
                onClick = { selectedOption = option }
            )
            Text(
                text = "$option"
            )
        }
    }
}
```



Checkbox

Checkbox

@Composable

fun Checkbox(
 checked: Boolean,
 onCheckedChange: ((Boolean) -> Unit)?,
 modifier: Modifier = Modifier,
 enabled: Boolean = true,
 colors: CheckboxColors = CheckboxDefaults.colors(),
 interactionSource: MutableInteractionSource? = null
)

Create a list of items for UI

Hold the set of currently-selected skills.

```
val skills = listOf("Android", "Kotlin", "Java", "Swift")
```

```
var selectedSkills by rememberSaveable { mutableStateOf(setOf<String>()) }
```

```

Column(modifier = modifier) {
    Column {
        skills.forEach {skill->
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.Start
            ) {
                Checkbox(
                    checked = selectedSkills.contains(skill),
                    onCheckedChange = {isChecked->
                        selectedSkills = if (isChecked) {
                            selectedSkills.plus( element = skill)
                        } else {
                            selectedSkills.minus( element = skill)
                        }
                    }
                )
                Text(
                    text = skill
                )
            }
        }
    }
}

```

//...

15:00

☐ Android

☐ Kotlin

☐ Java

☐ Swift

Submit

```

val context = LocalContext.current
val skills = listOf("Android", "Kotlin", "Java", "Swift")
var selectedSkills by rememberSaveable { mutableStateOf(value = se
Column(modifier = modifier) {
    Column {
        skills.forEach {skill->
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.Start
            ) {...}
        }
    }
    Button(
        onClick = {
            Toast.makeText(
                context,
                text = "Selected skills: ${selectedSkills}",
                duration = Toast.LENGTH_SHORT).show()
        },
        modifier = Modifier.align(Alignment.Start)
    ) {
        Text(text = "Submit")
    }
}

```

Add a Button

☐ Android
☐ Kotlin
☐ Java
☐ Swift

Submit